

phenotypes

2023-10-10

Cleaning Phenotypes

K. Uckele October 10, 2023

```
##
## Attaching package: 'dplyr'

## The following objects are masked from 'package:stats':
##
##   filter, lag

## The following objects are masked from 'package:base':
##
##   intersect, setdiff, setequal, union

## -- Attaching core tidyverse packages ----- tidyverse 2.0.0 --
## v forcats   1.0.0      v readr     2.1.5
## v ggplot2   3.5.2      v stringr  1.5.1
## v lubridate 1.9.4      v tibble   3.2.1
## v purrr     1.0.4      v tidyr    1.3.1
## -- Conflicts ----- tidyverse_conflicts() --
## x dplyr::filter() masks stats::filter()
## x dplyr::lag()    masks stats::lag()
## i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become errors
##
## Attaching package: 'EnvStats'
##
##
## The following objects are masked from 'package:stats':
##
##   predict, predict.lm
```

Red area (nectar guide data)

Load and filter data

```
redarea <- read_sheet(ss="19XH13K57mDi2BpMPrlNOR2w16VSswVOYP6WYKYJHvuqs", sheet="red_area")
```

```
## ! Using an auto-discovered, cached token.

## To suppress this message, modify your code or options to clearly consent to
## the use of a cached token.

## See gargle's "Non-interactive auth" vignette for more details:
## <https://gargle.r-lib.org/articles/non-interactive-auth.html>

## i The googlesheets4 package is using a cached token for 'kuckele@ucsc.edu'.
```

```
## v Reading from "F2_phenotypes".
## v Range 'red_area'.
#make a new column that makes a unique ID for each plant
redarea <- mutate(redarea, unique_ID = paste0(plant_type, "_", ID))

#exclude columns we don't need
redarea <- redarea %>% dplyr::select(-date, -photo_set, -plant_type, -ID, -labellum_photo, -stamen_photo)

#rename columns
redarea <- redarea %>% dplyr::rename("RALA" = "red_labellum",
                                     "RAST" = "red_stamen")

#take a quick look at data frame
redarea

## # A tibble: 238 x 3
##       RALA  RAST unique_ID
##   <dbl> <dbl> <chr>
## 1 0      0    39_2
## 2 0.002  0    39_3
## 3 63.4   4.92 39_5
## 4 54.3   12.3 39_5
## 5 71.4   3.6  39_6
## 6 109.   9.3  39_6
## 7 4.7    1.01 39_9
## 8 1.4    1.1  39_9
## 9 0      0    39_10
## 10 0     0    39_10
## # i 228 more rows
```

Collapse the replicate observations

```
#First, make a function to calculate the mode of categorical variables
#this function will output NA if there is a tie
Modes <- function(x) {
  ux <- unique(na.omit(x))
  tab <- tabulate(match(x, ux))
  if(sum(tab == max(tab)) == 2) {NA}
  else {ux[tab == max(tab)]}
}

#collapse the replicates by taking means
#first take the mean of continuous data
redarea_by_ID <- redarea %>% group_by(unique_ID) %>% reframe(
  tibble(
    across(where(is.double), \(x) mean(x, na.rm = TRUE)),
    across(where(is.character), Modes),
    across(where(is.factor), Modes)
  )
)
```

Identify potential outliers

```
traits <- setdiff(names(redarea_by_ID), "unique_ID")

result <- reduce(traits,          # loop over traits
  .init = list(data = redarea_by_ID, plots = list()),
  .f = function(acc, tr) {

    dat <- acc$data
    vec <- dat[[tr]]

    ## 1. Rosner test - how many to remove?
    k <- rosnerTest(vec, k = length(boxplot.stats(vec)$out))$n.outliers

    ## 2. pick k single most-extreme values
    outs <- boxplot.stats(vec)$out
    center <- median(vec, na.rm = TRUE)
    extreme <- outs[order(abs(outs - center), decreasing = TRUE)][seq_len(k)]

    message(sprintf("Trait %s - %d extreme values flagged (unique_IDs starting with 'P' and 'F1' kept)"
      tr, length(extreme)))

    ## 3. save BEFORE plot
    p_before <- ggplot(dat, aes(x = "", y = .data[[tr]])) +
      geom_boxplot(fill = "#0c4c8a") +
      theme_minimal() +
      ggtitle(glue::glue("{tr}: before"))

    ## 4. remove those extremes unless ID starts with P or F1
    dat2 <- dat %>%
      mutate(
        {{ tr }} := if_else(
          .data[[tr]] %in% extreme & !grepl("^(P|F1)", unique_ID),
          NA_real_,
          .data[[tr]]
        )
      )

    ## 5. AFTER plot
    p_after <- ggplot(dat2, aes(x = "", y = .data[[tr]])) +
      geom_boxplot(fill = "#0c4c8a") +
      theme_minimal() +
      ggtitle(glue::glue("{tr}: after"))

    ## 6. sanity-check: Rosner should find zero now
    if (k > 0) {
      remaining <- rosnerTest(dat2[[tr]],
        k = length(extreme))$n.outliers

      if (remaining > 0)
        warning(glue::glue("{tr}: {remaining} outlier(s) still present"))
    }

    ## 7. store updated data and plots
    acc$data <- dat2
  })
```

```

acc$plots[[tr]] <- p_before | p_after
acc
})

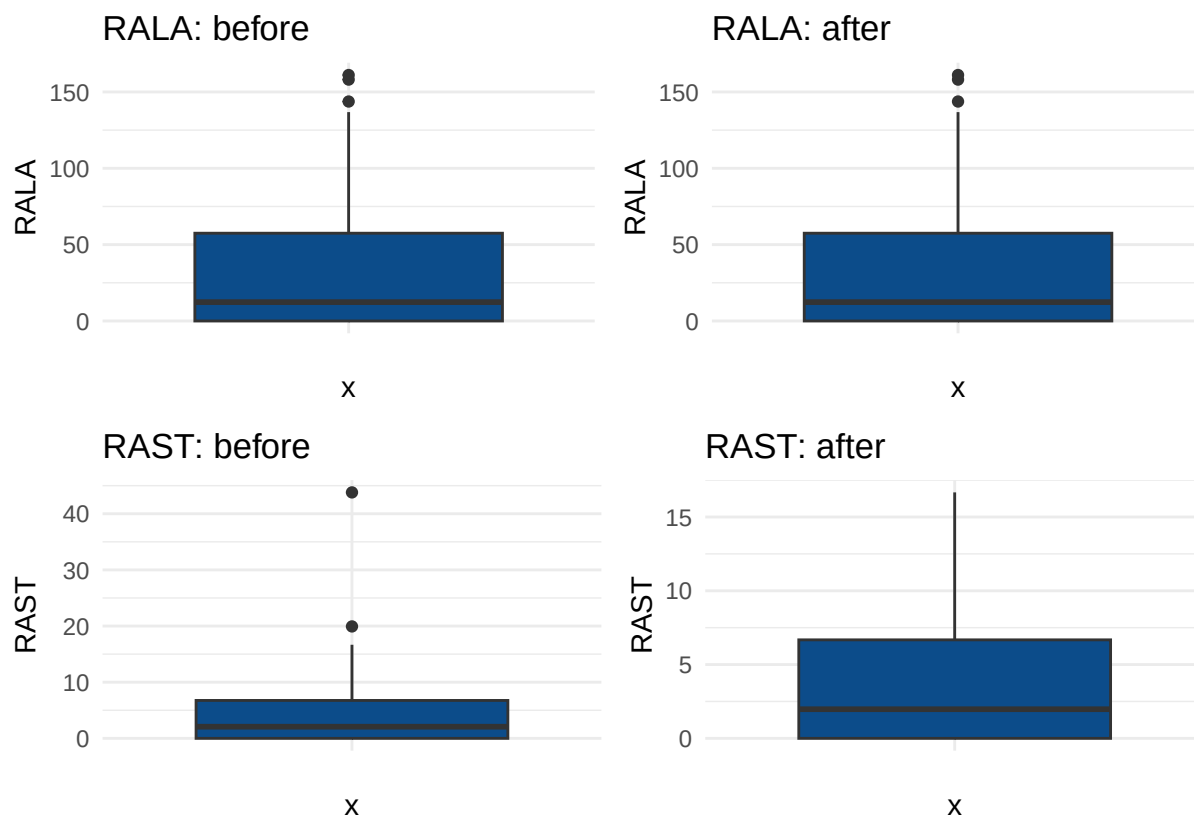
## Trait RALA - 0 extreme values flagged (unique_IDs starting with 'P' and 'F1' kept)
## Trait RAST - 2 extreme values flagged (unique_IDs starting with 'P' and 'F1' kept)
## Warning in rosnerTest(dat2[[tr]], k = length(extreme)): 2 observations with
## NA/NaN/Inf in 'x' removed.

redarea_by_ID <- result$data
plots <- result$plots # list of before/after panels

# combine all plots into grid
wrap_plots(plots, nrow = length(traits)) # grid of all traits

## Warning: Removed 2 rows containing non-finite outside the scale range
## (`stat_boxplot()`).

```



Inflorescence-level data

Load and filter data

```

#get access to google sheets
gs4_auth(email = "kuckele@ucsc.edu")

inflor <- read_sheet(
  ss = "19XH13K57mDi2BpMPrlNOR2w16VSvVOYP6WYKYJHvuqs",

```

```

sheet = "inflorescences",
col_types = "?????l?l"
) %>%
  # add your extra wrangling steps
  mutate(unique_ID = paste0(plant_type, "_", ID)) %>%
  select(-date, -rep, -plant_type, -ID, -typeID) %>%
  # convert the two visibility flags to factors
  #mutate(across(c(visible_EFnectar, visible_guides), as.factor)) %>%
  dplyr::rename("INFA" = "infl_angle",
                "VEFN" = "visible_EFnectar",
                "CAL" = "callus_length",
                "VNG" = "visible_guides")

```

```
## v Reading from "F2_phenotypes".
```

```
## v Range 'inflorescences'.
```

```

#take a quick look at data frame
inflor

```

```

## # A tibble: 606 x 5
##   INFA VEFN    CAL VNG  unique_ID
##   <dbl> <lgl> <dbl> <lgl> <chr>
## 1    25 TRUE   4.89 FALSE 39_2
## 2    20 TRUE   3.9  FALSE 39_2
## 3    70 TRUE   4.1  FALSE 39_2
## 4   100 TRUE   5.27 FALSE 39_2
## 5    18 TRUE   4.4  FALSE 39_3
## 6    15 TRUE   5.19 FALSE 39_3
## 7     0 TRUE   3.48 FALSE 39_3
## 8    37 TRUE   4.09 TRUE  39_4
## 9    55 FALSE  4.7  FALSE 39_5
## 10    5 TRUE   4.81 TRUE  39_5
## # i 596 more rows

```

Collapse the replicate observations

```

#collapse the replicates by taking means and modes
#take the mean of continuous data and the mode of the factors and characters
inflor_by_ID <- inflor %>%
  group_by(unique_ID) %>%
  summarise(
    across(where(is.double), ~ mean(.x, na.rm = TRUE)), # numeric columns
    across(where(is.character), Modes),                # your custom mode fn
    across(where(is.logical), Modes),                  # your custom mode fn
    .groups = "drop"                                    # ungroups automatically
  )

```

Fill in VNG based on red area data

```

# Join the two data frames by the unique_ID column
inflor_by_ID <- inflor_by_ID %>%
  left_join(redarea_by_ID, by = "unique_ID") %>%
  mutate(VNG = case_when(

```

```

is.na(VNG) & (RAST > 0 | RALA > 0) ~ TRUE,
is.na(VNG) & RAST == 0 & RALA == 0 ~ FALSE,
TRUE ~ VNG # Keep the original value if the condition is not met
)) %>%
# Remove the columns from redarea_by_ID if no longer needed
select(-RAST, -RALA)

```

Identify potential outliers

```

traits <- setdiff(names(inflor_by_ID), c("unique_ID", "VEFN", "VNG"))

result <- reduce(traits, # loop over traits
  .init = list(data = inflor_by_ID, plots = list()),
  .f = function(acc, tr) {

    dat <- acc$data
    vec <- dat[[tr]]

    ## 1. Rosner test - how many to remove?
    k <- rosnerTest(vec, k = length(boxplot.stats(vec)$out))$n.outliers

    ## 2. pick k single most-extreme values
    outs <- boxplot.stats(vec)$out
    center <- median(vec, na.rm = TRUE)
    extreme <- outs[order(abs(outs - center), decreasing = TRUE)][seq_len(k)]

    message(sprintf("\n\nTrait %s - %d extreme values flagged (unique_IDs starting with 'P' and 'F1' key)
                    tr, length(extreme)))

    ## 3. save BEFORE plot
    p_before <- ggplot(dat, aes(x = "", y = .data[[tr]])) +
      geom_boxplot(fill = "#0c4c8a") +
      theme_minimal() +
      ggtitle(glue::glue("{tr}: before"))

    ## 4. remove those extremes unless ID starts with P or F1
    dat2 <- dat %>%
      mutate(
        {{ tr }} := if_else(
          .data[[tr]] %in% extreme & !grepl("^(P|F1)", unique_ID),
          NA_real_,
          .data[[tr]]
        )
      )

    ## 5. AFTER plot
    p_after <- ggplot(dat2, aes(x = "", y = .data[[tr]])) +
      geom_boxplot(fill = "#0c4c8a") +
      theme_minimal() +
      ggtitle(glue::glue("{tr}: after"))

    ## 6. sanity-check: Rosner should find zero now
    if (k > 0) {

```

```

# remove the parent and F1 observations from the data before testing
vec_keep <- dat %>%
  filter(!grepl("^(P|F1)", unique_ID)) %>%
  pull({{ tr }})

remaining <- rosnerTest(vec_keep,
                        k = length(extreme))$n.outliers
if (remaining > 0)
  warning(glue::glue("{tr}: {remaining} outlier(s) still present"))
}

## 7. store updated data and plots
acc$data <- dat2
acc$plots[[tr]] <- p_before | p_after
acc
})

## Warning in rosnerTest(vec, k = length(boxplot.stats(vec)$out)): The true Type I error may be larger
## Although the help file for 'rosnerTest' has a table with information
## on the estimated Type I error level,
## simulations were not run for k > 10 or k > floor(n/2).

##
##
## Trait INFA - 1 extreme values flagged (unique_IDs starting with 'P' and 'F1' kept)
## Warning in rosnerTest(vec, k = length(boxplot.stats(vec)$out)): 1 observations
## with NA/NaN/Inf in 'x' removed.

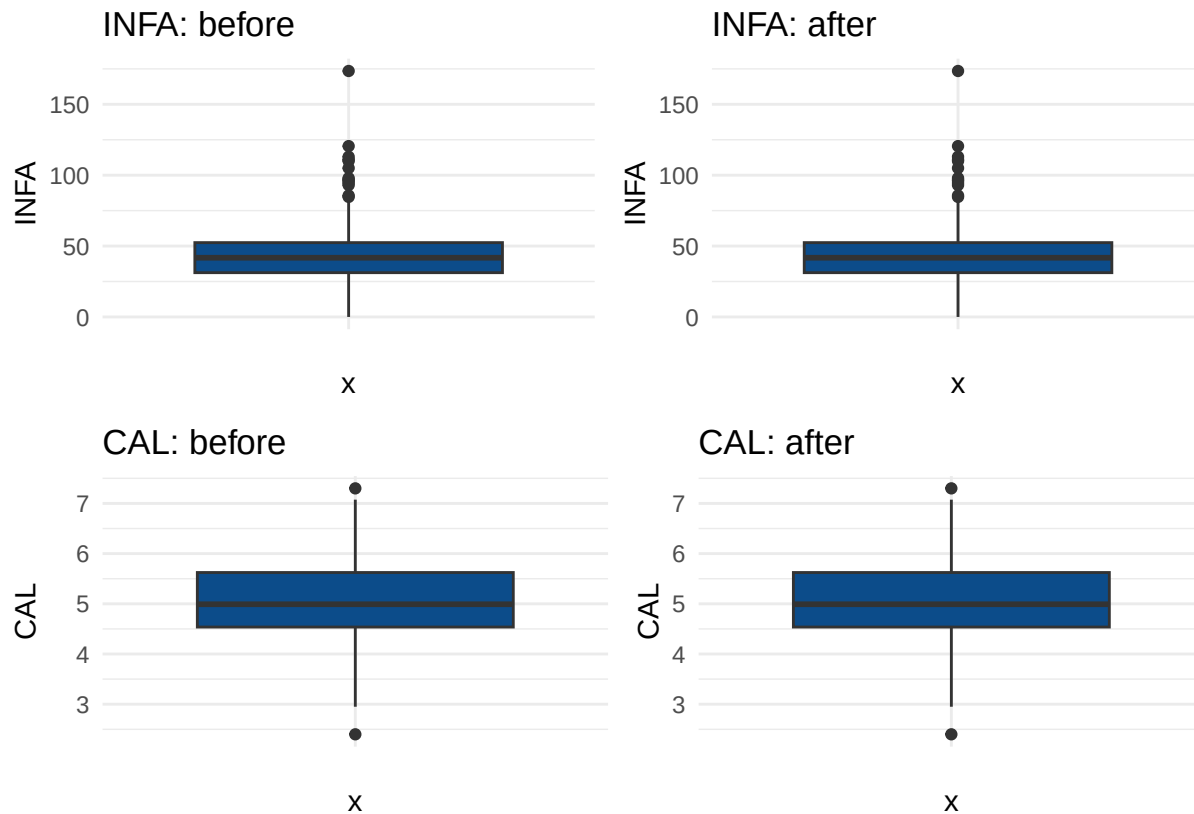
##
##
## Trait CAL - 0 extreme values flagged (unique_IDs starting with 'P' and 'F1' kept)
inflor_by_ID <- result$data
plots <- result$plots # list of before/after panels

# combine all plots into grid
wrap_plots(plots, nrow = length(traits)) # grid of all traits

## Warning: Removed 1 row containing non-finite outside the scale range
## (`stat_boxplot()`).

## Warning: Removed 1 row containing non-finite outside the scale range
## (`stat_boxplot()`).

```



Flower morphology data

Load and filter data

```
flo_morph <- read_sheet(ss="19XH3K57mDi2BpMPrlNOR2w16VSwV0YP6WYKYJHvuqs", sheet="flower_morphology")

## v Reading from "F2_phenotypes".
## v Range 'flower_morphology'.

#make a new column that makes a unique ID for each plant
flo_morph <- mutate(flo_morph, unique_ID = paste0(plant_type, "_", ID))

#exclude columns we don't need
flo_morph <- flo_morph %>% dplyr::select(-date, -plant_type, -ID, -rep)

#rename columns
flo_morph <- flo_morph %>% dplyr::rename("COL" = "Corolla_Length",
                                         "COLL" = "Corolla_Lobe_Length",
                                         "STAE" = "Stamen_exsertion",
                                         "TUA" = "tube_angle",
                                         "STATL" = "stamen_tip",
                                         "LABL" = "Labellum_Length",
                                         "LABW" = "Labellum_Width",
                                         "CLL" = "Labellum_lobe",
                                         "STAL" = "Stamen_Length",
                                         "STAW" = "Stamen_width",
                                         "ANL" = "Anther_Length",
```

```

      "ANW" = "Anther_width",
      "STIW" = "Stigma_Width",
      "STYL" = "Style_length")

#quick look at data
flo_morph

## # A tibble: 767 x 15
##   COL COLL STAE TUA STATL LABL LABW CLL STAL STAW ANL ANW STIW
##   <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl>
## 1 48.8 30.5 -2.3 24 8.2 54.4 29.6 7.5 52.4 9.4 6.4 2.7 2.6
## 2 52.1 35 -1.4 27 8.5 57.4 28.3 5.3 55.9 9.4 6.8 2.7 3
## 3 51.4 30.4 -2.4 7 8 58 25.5 6.6 55.6 9.4 6.6 2.8 2.7
## 4 45.7 27.4 -2.9 21 7.2 52.7 27.7 5.6 49.8 9.3 6.6 2.4 2.7
## 5 46.5 26.6 -2.1 5 6.5 53.3 26.9 6.7 51.2 8.6 6.7 2.4 2.8
## 6 47.3 28.3 -3.7 21 6.9 53.2 27.9 5.6 49.5 9.8 6.5 3.1 2.5
## 7 46.3 26.4 -1.9 13 7 50.8 22.3 4.2 48.9 8.6 6.4 2.4 2.3
## 8 50.6 29.6 -2.9 24 11.1 57.6 31.1 5 54.7 9.3 6.4 2.8 2.6
## 9 49.3 27.9 -3.1 11 11 58.1 31.2 6.9 55 9.5 6.8 2.8 3.2
## 10 51.9 32.7 -2.4 10 11.5 58.5 31.4 9 56.1 9.4 6.9 2.8 2.9
## # i 757 more rows
## # i 2 more variables: STYL <dbl>, unique_ID <chr>

```

Collapse the replicate observations

```

flo_morph_by_ID <- flo_morph %>% group_by(unique_ID) %>% reframe(
  tibble(
    across(where(is.double), \(x) mean(x, na.rm = TRUE)),
    across(where(is.character), Modes),
    across(where(is.factor), Modes)
  )
)

```

Identify potential outliers

```

traits <- setdiff(names(flo_morph_by_ID), c("unique_ID"))

result <- reduce(traits, # loop over traits
  .init = list(data = flo_morph_by_ID, plots = list()),
  .f = function(acc, tr) {

    dat <- acc$data

    #vec <- dat[[tr]]

    vec <- dat %>% # rows that CAN be tested/removed
      filter(!grepl("^P|F1", unique_ID)) %>%
      pull({{ tr }})

    ## 1. Rosner test - how many to remove?
    if (length(boxplot.stats(vec)$out) == 0) {
      test.k <- 1
    } else {

```

```

    test.k <- length(boxplot.stats(vec)$out)
  }

  k <- rosnerTest(vec, k = test.k)$n.outliers

  ## 2. pick k single most-extreme values
  outs <- boxplot.stats(vec)$out
  center <- median(vec, na.rm = TRUE)
  extreme <- outs[order(abs(outs - center), decreasing = TRUE)][seq_len(k)]

  message(sprintf("\n\nTrait %s - %d extreme values flagged (unique_IDs starting with 'P' and 'F1' kept)
                  tr, length(extreme)))

  ## 3. save BEFORE plot
  p_before <- ggplot(dat, aes(x = "", y = .data[[tr]])) +
    geom_boxplot(fill = "#0c4c8a") +
    theme_minimal() +
    ggtitle(glue::glue("{tr}: before"))

  ## 4. remove those extremes unless ID starts with P or F1
  dat2 <- dat %>%
    mutate(
      {{ tr }} := if_else(
        .data[[tr]] %in% extreme & !grepl("^P|F1", unique_ID),
        NA_real_,
        .data[[tr]]
      )
    )

  ## 5. AFTER plot
  p_after <- ggplot(dat2, aes(x = "", y = .data[[tr]])) +
    geom_boxplot(fill = "#0c4c8a") +
    theme_minimal() +
    ggtitle(glue::glue("{tr}: after"))

  ## 6. sanity-check: Rosner should find zero now
  if (k > 0) {

    # remove the parent and F1 observations from the data before testing
    vec_keep <- dat2 %>%
      filter(!grepl("^P|F1", unique_ID)) %>%
      pull({{ tr }})

    remaining <- rosnerTest(vec_keep,
                           k = length(extreme))$n.outliers
    if (remaining > 0)
      warning(glue::glue("{tr}: {remaining} outlier(s) still present"))
  }

  ## 7. store updated data and plots
  acc$data <- dat2
  acc$plots[[tr]] <- p_before | p_after
  acc

```

```

})

##
##
## Trait COL - 1 extreme values flagged (unique_IDs starting with 'P' and 'F1' kept)
## Warning in rosnerTest(vec_keep, k = length(extreme)): 1 observations with
## NA/NaN/Inf in 'x' removed.
##
##
## Trait COLL - 0 extreme values flagged (unique_IDs starting with 'P' and 'F1' kept)
##
##
## Trait STAE - 0 extreme values flagged (unique_IDs starting with 'P' and 'F1' kept)
##
##
## Trait TUA - 3 extreme values flagged (unique_IDs starting with 'P' and 'F1' kept)
## Warning in rosnerTest(vec_keep, k = length(extreme)): 3 observations with
## NA/NaN/Inf in 'x' removed.
##
##
## Trait STATL - 0 extreme values flagged (unique_IDs starting with 'P' and 'F1' kept)
##
##
## Trait LABL - 0 extreme values flagged (unique_IDs starting with 'P' and 'F1' kept)
##
##
## Trait LABW - 0 extreme values flagged (unique_IDs starting with 'P' and 'F1' kept)
##
##
## Trait CLL - 1 extreme values flagged (unique_IDs starting with 'P' and 'F1' kept)
## Warning in rosnerTest(vec_keep, k = length(extreme)): 1 observations with
## NA/NaN/Inf in 'x' removed.
##
##
## Trait STAL - 0 extreme values flagged (unique_IDs starting with 'P' and 'F1' kept)
##
##
## Trait STAW - 1 extreme values flagged (unique_IDs starting with 'P' and 'F1' kept)
## Warning in rosnerTest(vec_keep, k = length(extreme)): 1 observations with
## NA/NaN/Inf in 'x' removed.
##
##
## Trait ANL - 1 extreme values flagged (unique_IDs starting with 'P' and 'F1' kept)
## Warning in rosnerTest(vec_keep, k = length(extreme)): 1 observations with
## NA/NaN/Inf in 'x' removed.

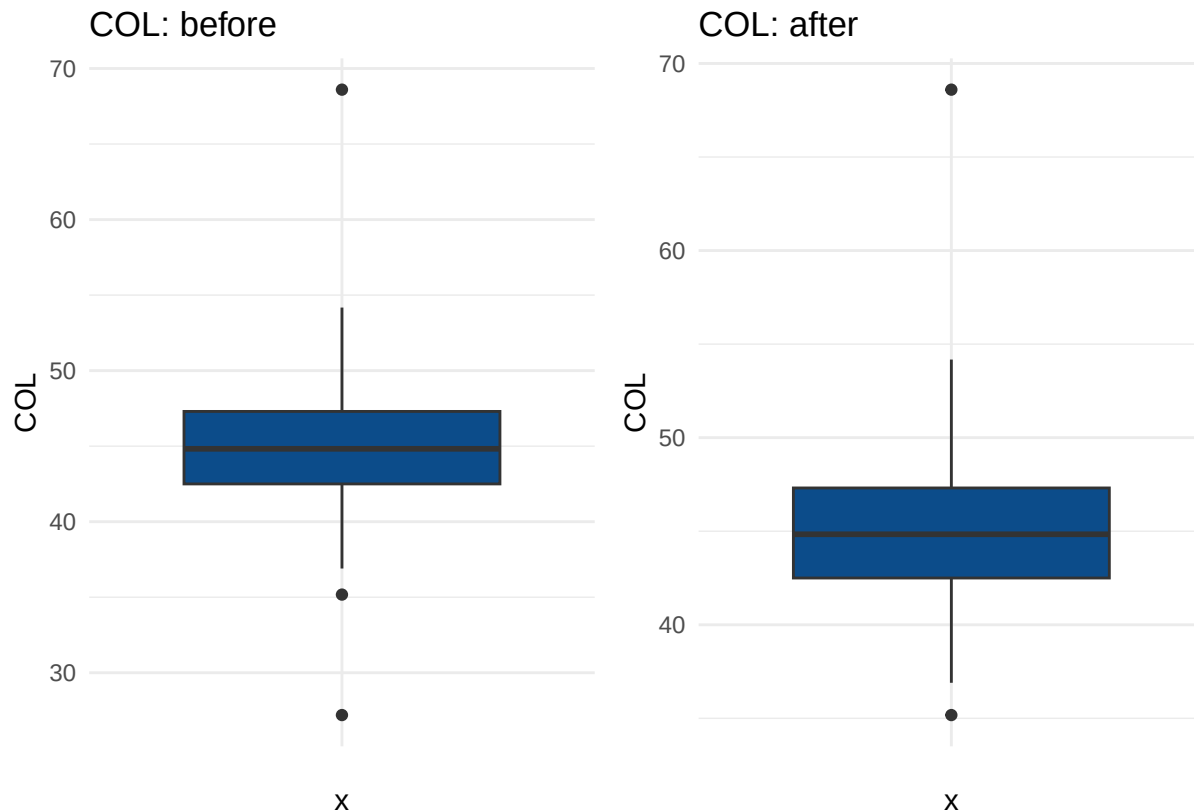
```

```
##
##
## Trait ANW - 0 extreme values flagged (unique_IDs starting with 'P' and 'F1' kept)
##
##
## Trait STIW - 1 extreme values flagged (unique_IDs starting with 'P' and 'F1' kept)
## Warning in rosnerTest(vec_keep, k = length(extreme)): 1 observations with
## NA/NaN/Inf in 'x' removed.
##
##
## Trait STYL - 0 extreme values flagged (unique_IDs starting with 'P' and 'F1' kept)
flo_morph_by_ID <- result$data
plots <- result$plots      # list of before/after panels

# plot each trait
plots[1]
```

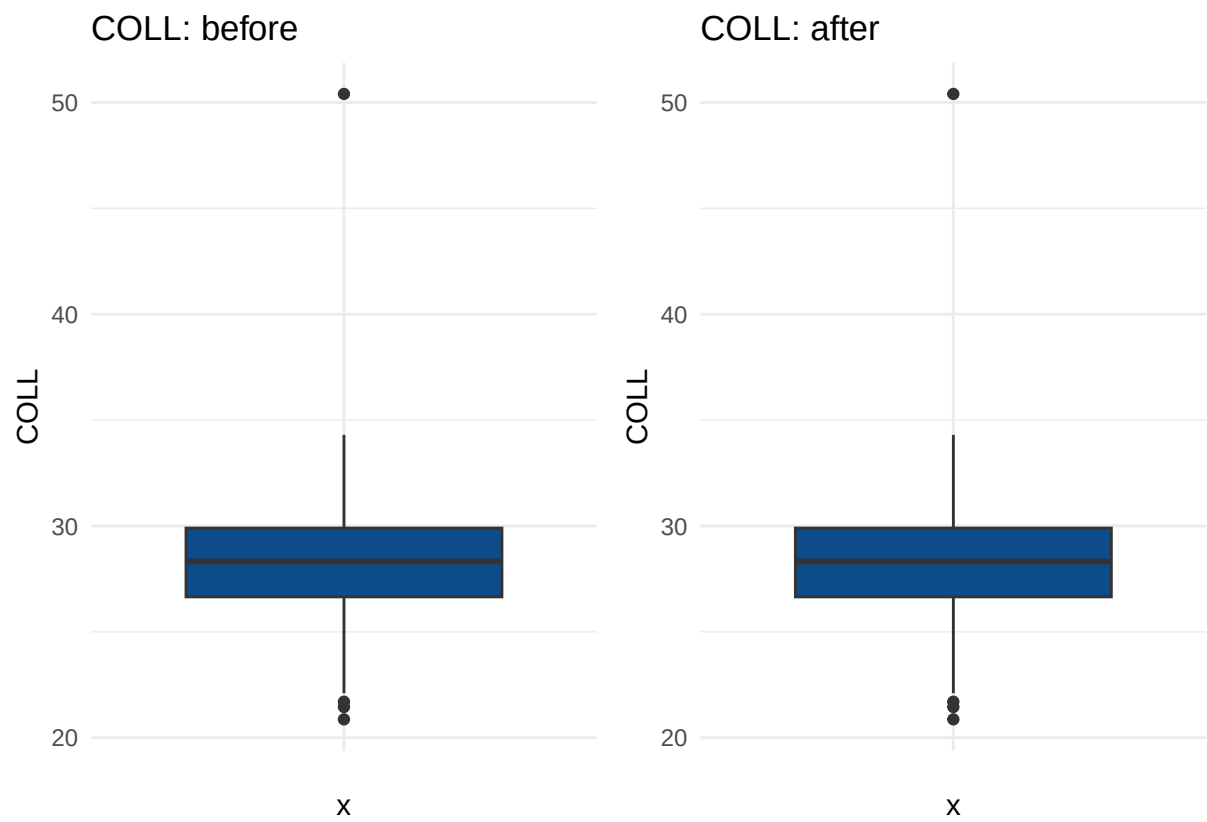
```
## $COL
```

```
## Warning: Removed 1 row containing non-finite outside the scale range
## (`stat_boxplot()`).
```



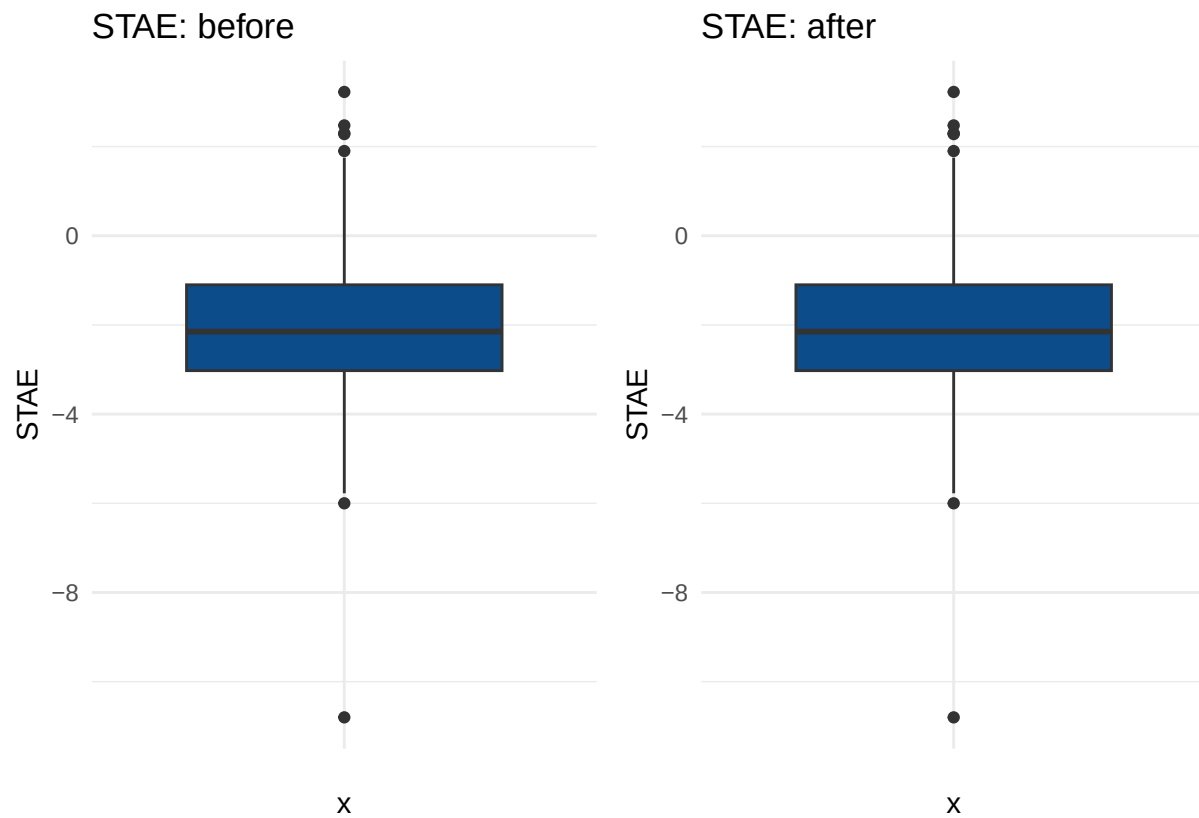
```
plots[2]
```

```
## $COLL
```



```
plots[3]
```

```
## $STAE
```

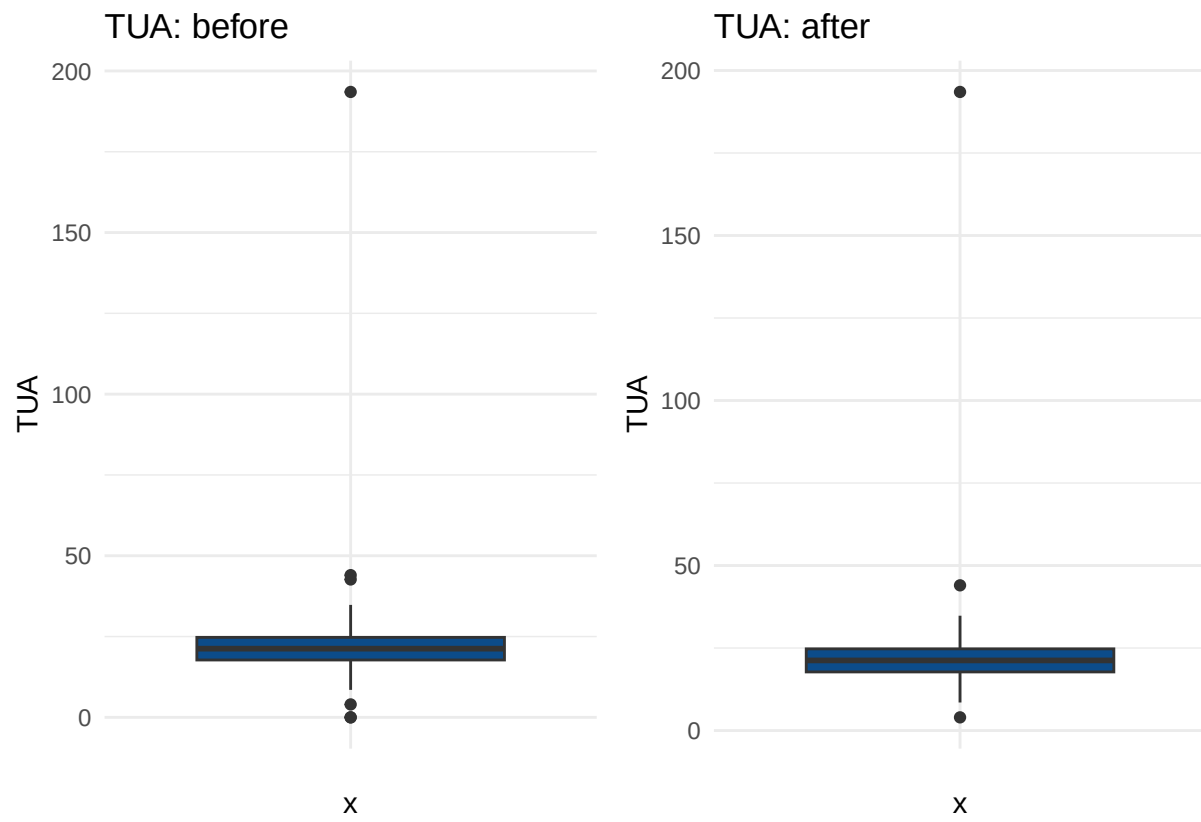


```
plots[4]
```

```
## $TUA
```

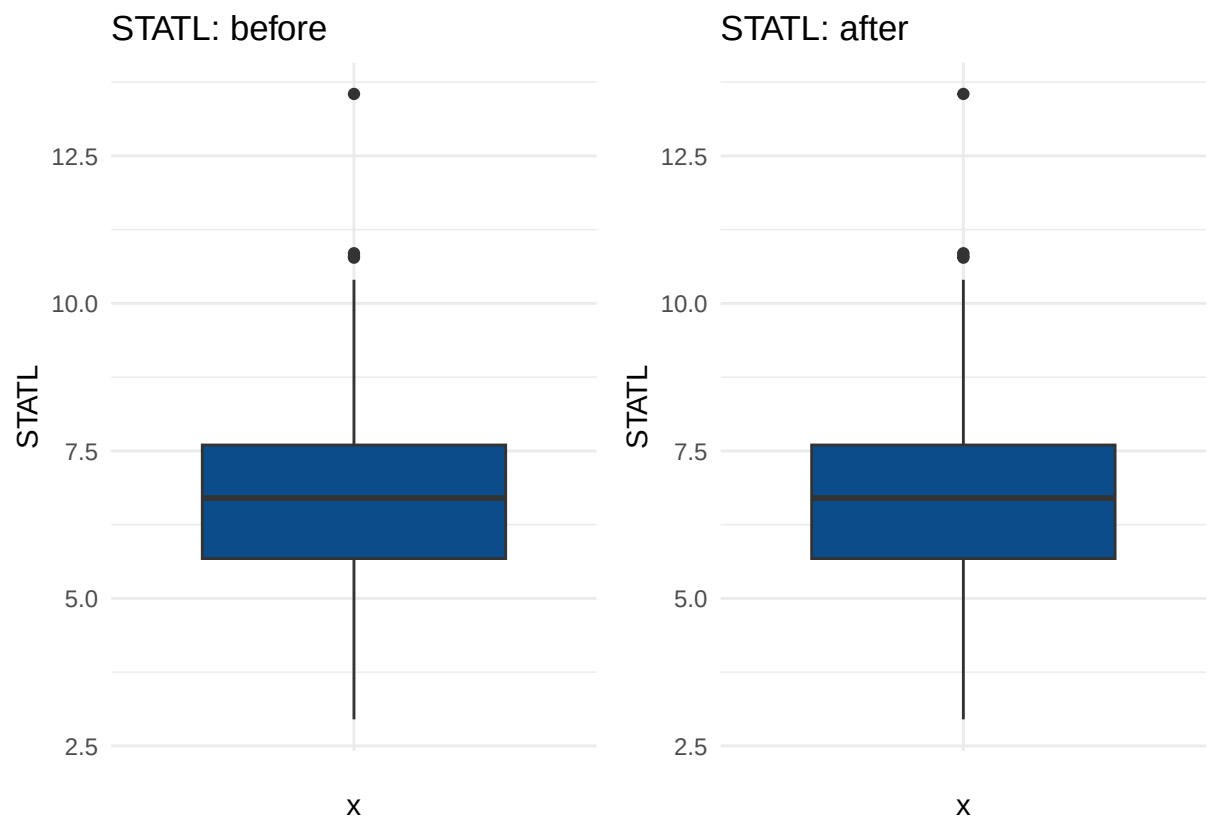
```
## Warning: Removed 3 rows containing non-finite outside the scale range
```

```
## (`stat_boxplot()`).
```



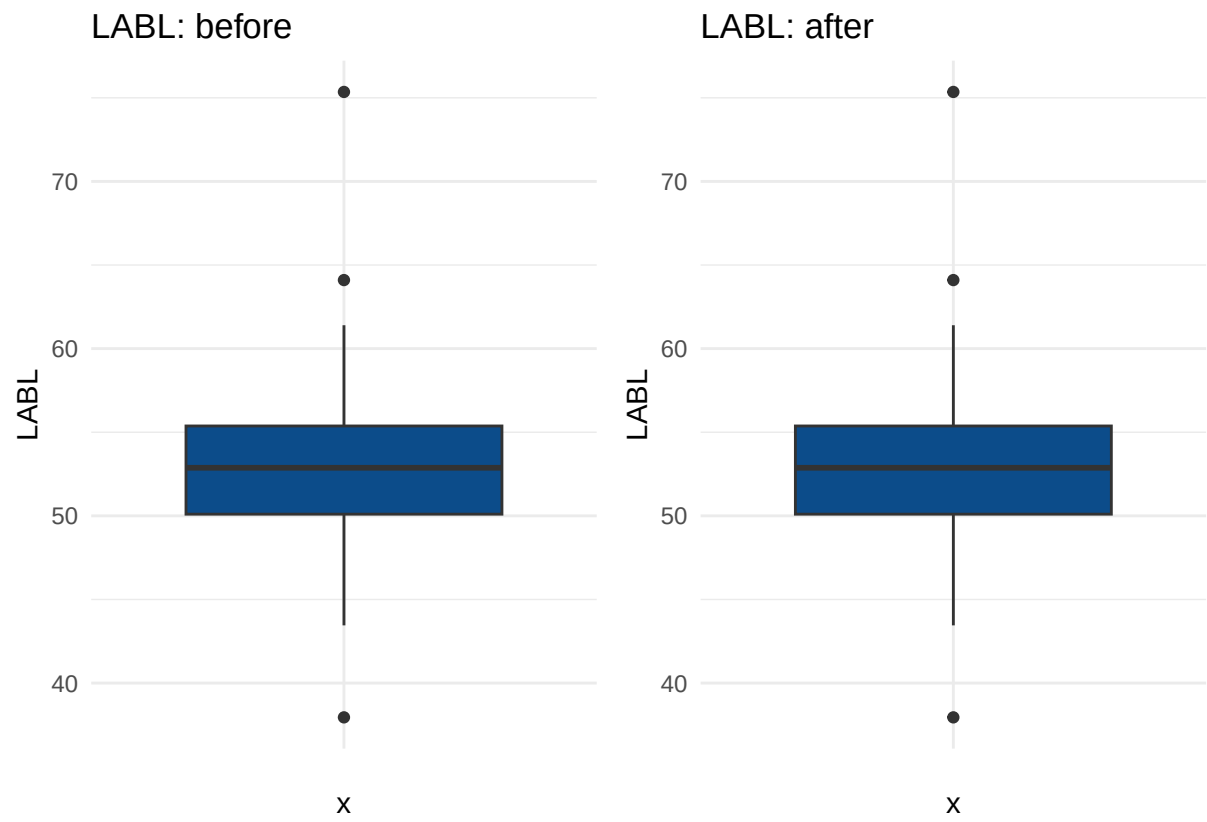
```
plots[5]
```

```
## $STATL
```



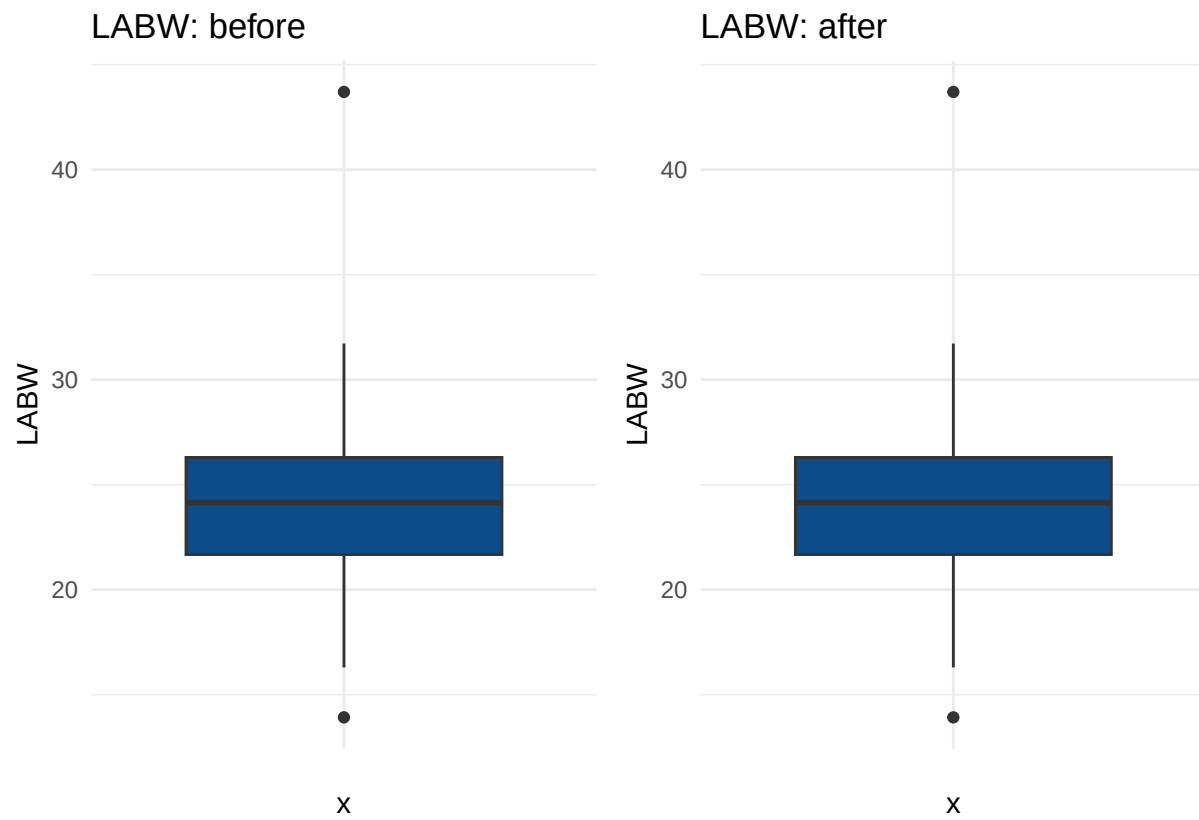
```
plots[6]
```

```
## $LABL
```



```
plots[7]
```

```
## $LABW
```

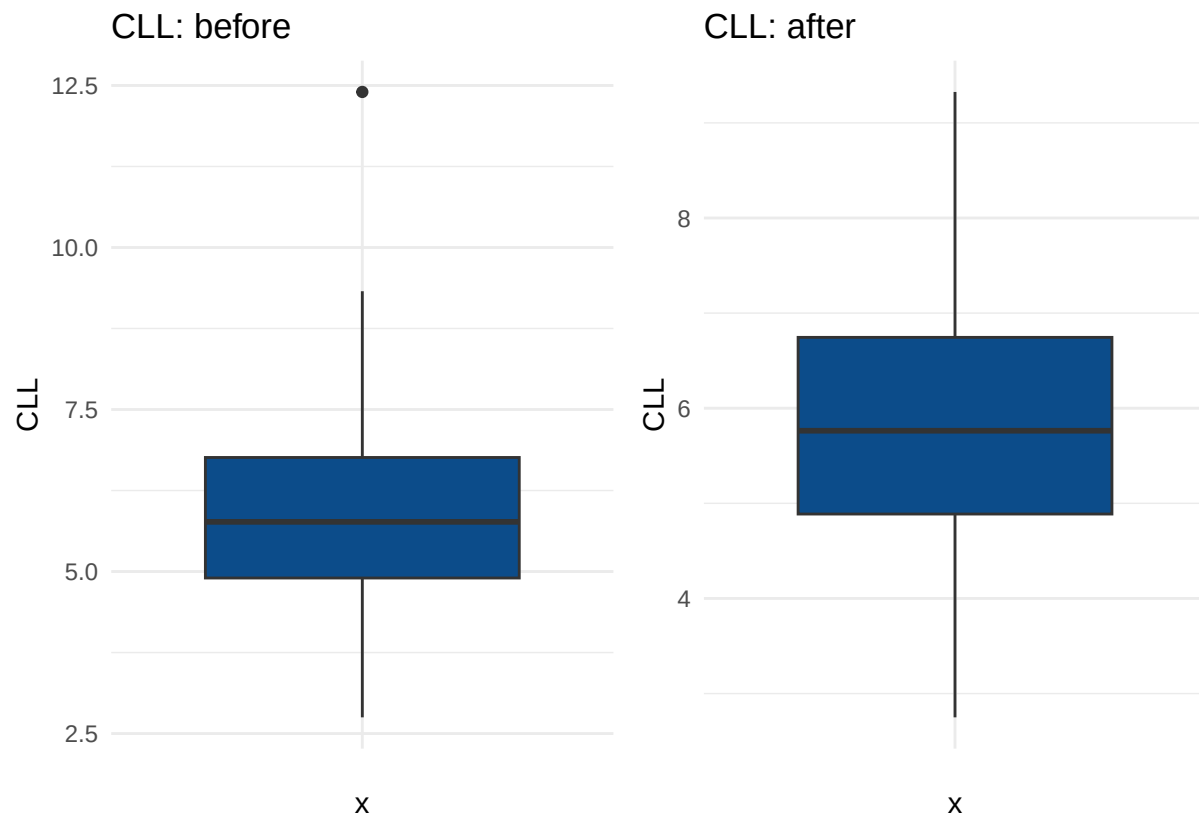


```
plots[8]
```

```
## $CLL
```

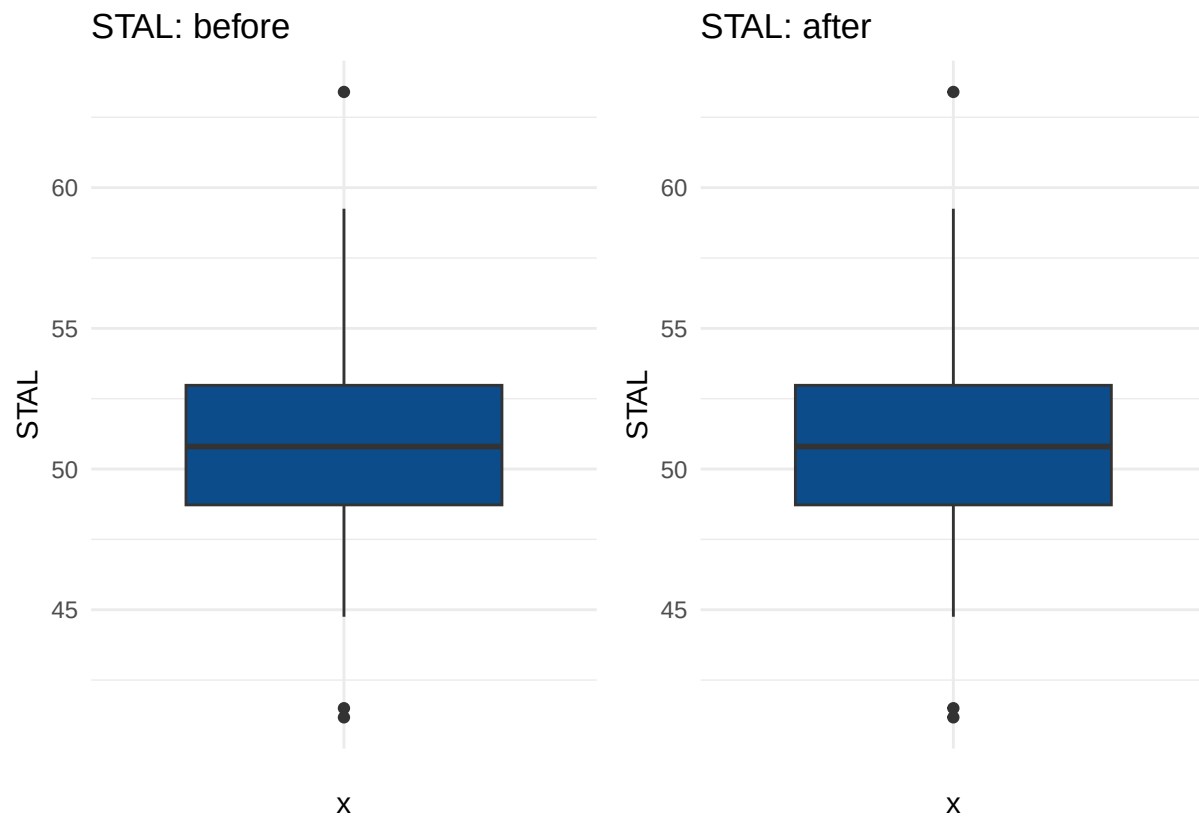
```
## Warning: Removed 1 row containing non-finite outside the scale range
```

```
## (`stat_boxplot()`).
```



```
plots[9]
```

```
## $STAL
```

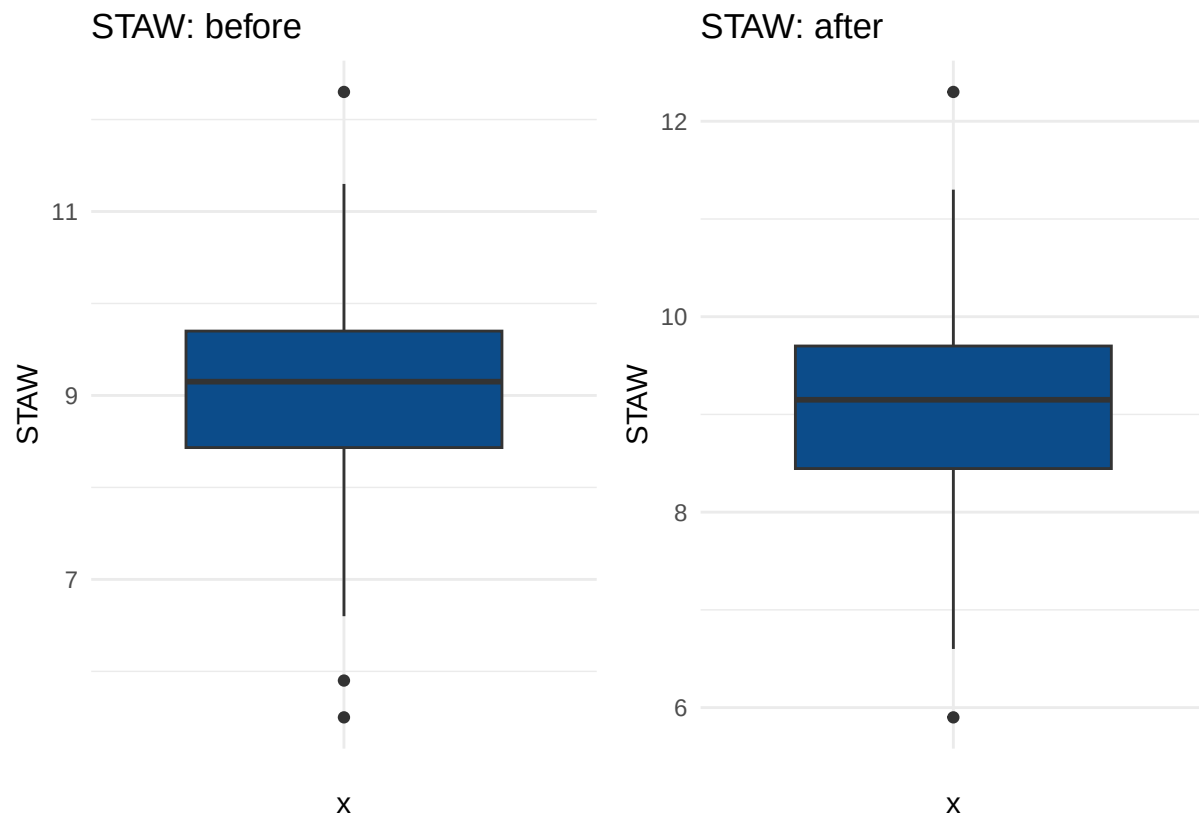


```
plots[10]
```

```
## $STAW
```

```
## Warning: Removed 1 row containing non-finite outside the scale range
```

```
## (`stat_boxplot()`).
```

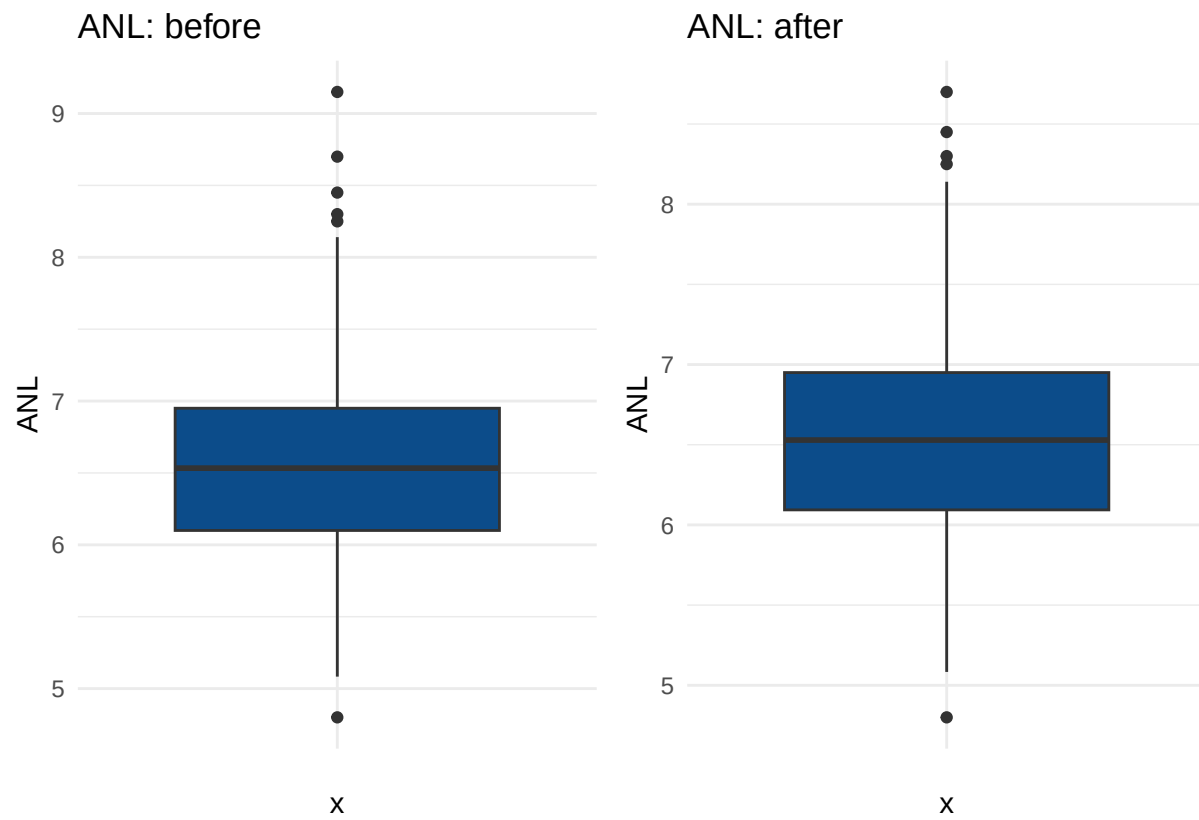


```
plots[11]
```

```
## $ANL
```

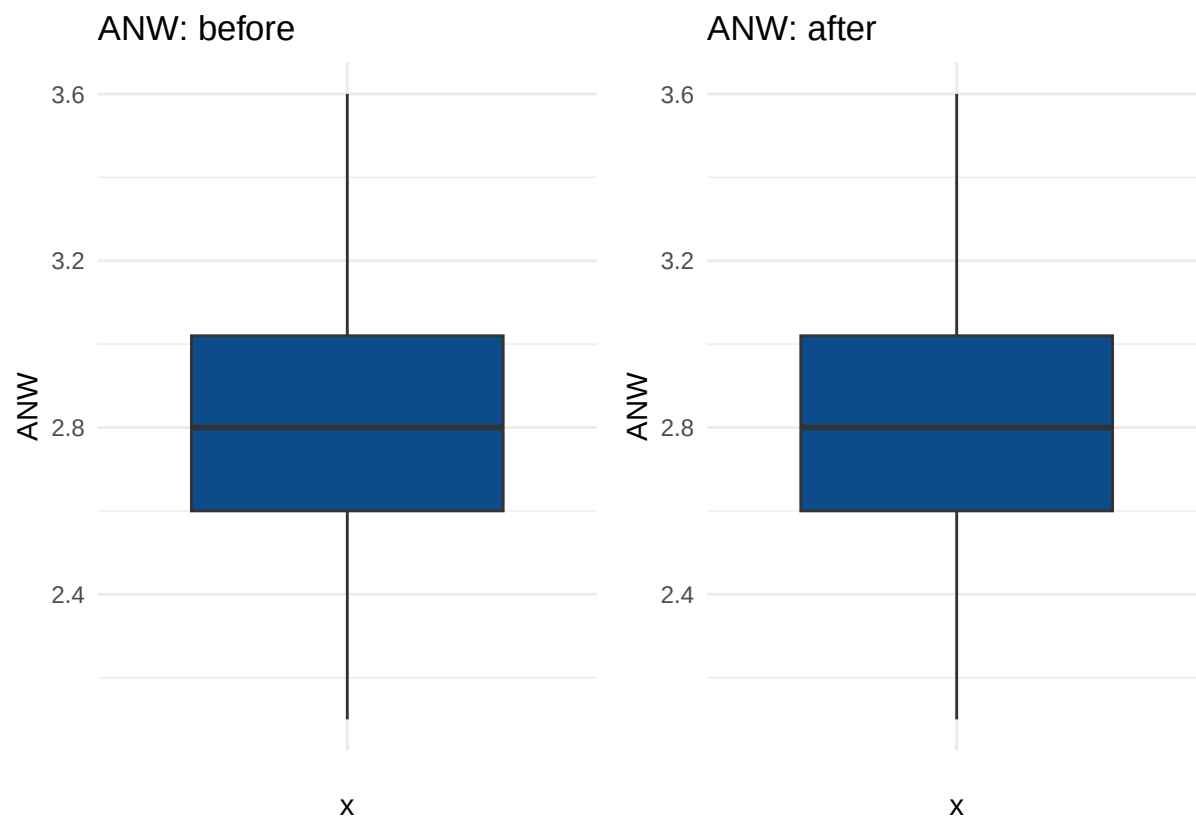
```
## Warning: Removed 1 row containing non-finite outside the scale range
```

```
## (`stat_boxplot()`).
```



```
plots[12]
```

```
## $ANW
```

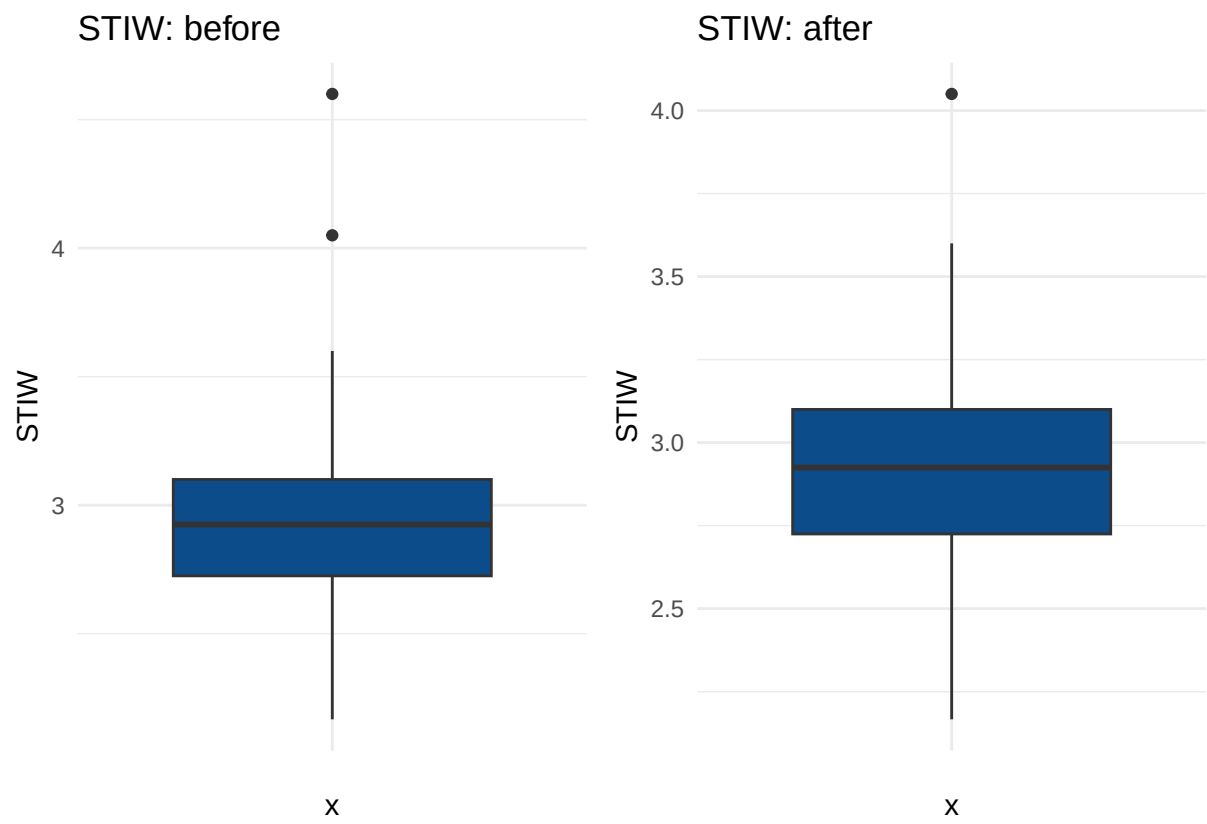


```
plots[13]
```

```
## $STIW
```

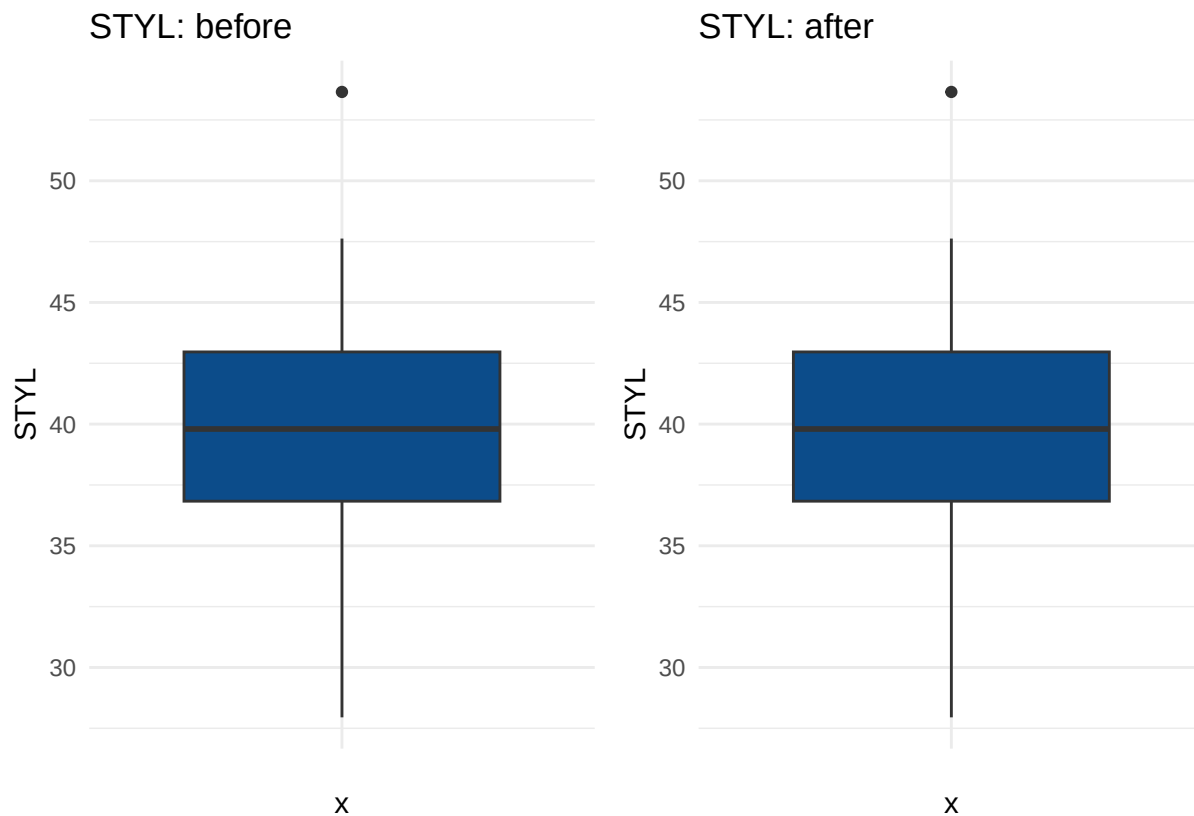
```
## Warning: Removed 1 row containing non-finite outside the scale range
```

```
## (`stat_boxplot()`).
```



```
plots[14]
```

```
## $STYL
```



Nectar data

Load and filter data

```
nectar <- read_sheet(ss="19XH13K57mDi2BpMPPr1NOR2w16VSwVOYP6WYKYJHvuqs", sheet="nectar")
```

```
## v Reading from "F2_phenotypes".
```

```
## v Range 'nectar'.
```

```
brix_density <- read_csv("~/Dropbox/Costus/costus-genetic-mapping/phenotype/scripts/brix_density_table.csv")
```

```
#make a new column that makes a unique ID for each plant
```

```
nectar <- mutate(nectar, unique_ID = paste0(plant_type, "_", ID))
```

```
#exclude columns we don't need
```

```
nectar <- nectar %>% dplyr::select(-year, -date, -plant_type, -ID, -rep, -fl_capsize, -fl_nectarlength_mL)
```

```
# calculate mass of sucrose in solution
```

```
EFN_percent_sucrose_rounded <- round(nectar$EFN_percent_sucrose)
```

```
densities <- c()
```

```
for (i in 1:length(EFN_percent_sucrose_rounded)) {
  density <- brix_density$Density[brix_density$Brix == EFN_percent_sucrose_rounded[i]]
  densities <- c(densities, density)
}
```

```
EFN_H2Ovolume_mL = nectar$EFN_H2Ovolume_uL / 1000
```

```

mass_solution <- EFN_H2Ovolume_mL * densities
mass_sugar_grams <- (EFN_percent_sucrose_rounded/100) * mass_solution
nectar$mass_sugar_mg <- mass_sugar_grams * 1000

#exclude columns we don't need
nectar <- nectar %>% dplyr::select(-EFN_H2Ovolume_uL, -EFN_percent_sucrose)

#rename columns
nectar <- nectar %>% dplyr::rename("VFN" = "fl_nectarvolume_uL", # volume floral nectar
                                "FNESC" = "fl_percent_sucrose", # percent sucrose floral nectar
                                "EFNESC" = "mass_sugar_mg") # mass of sucrose in mg

#quick look at data
nectar

```

```

## # A tibble: 1,031 x 4
##       VFN  FNESC unique_ID EFNESC
##   <dbl> <dbl> <chr>      <dbl>
## 1 13.8   32.5 39_2         5.46
## 2 14.9   27.5 39_2         2.45
## 3 20.8   28.5 39_2         2.03
## 4 21.2   24.5 39_2         1.21
## 5 18.1    32 39_2         7.71
## 6 13.3   33.5 39_3         3.29
## 7  5.9    31 39_3         3.72
## 8  1.78  16.5 39_3          0
## 9 16.2    36 39_3         6.80
## 10 11.1   36 39_3         2.45
## # i 1,021 more rows

```

Collapse the replicate observations

```

nectar_by_ID <- nectar %>% group_by(unique_ID) %>% reframe(
  tibble(
    across(where(is.double), \(x) mean(x, na.rm = TRUE)),
    across(where(is.character), Modes),
    across(where(is.factor), Modes)
  )
)

```

Identify potential outliers

```

traits <- setdiff(names(nectar_by_ID), c("unique_ID"))

result <- reduce(traits, # loop over traits
  .init = list(data = nectar_by_ID, plots = list()),
  .f = function(acc, tr) {

    dat <- acc$data

    #vec <- dat[[tr]]

    vec <- dat %>% # rows that CAN be tested/removed

```

```

  filter(!grepl("^(P|F1)", unique_ID)) %>%
  pull({{ tr }})

## 1. Rosner test - how many to remove?
if (length(boxplot.stats(vec)$out) == 0) {
  test.k <- 1
} else {
  test.k <- length(boxplot.stats(vec)$out)
}

k <- rosnerTest(vec, k = test.k)$n.outliers

## 2. pick k single most-extreme values
outs <- boxplot.stats(vec)$out
center <- median(vec, na.rm = TRUE)
extreme <- outs[order(abs(outs - center), decreasing = TRUE)][seq_len(k)]

message(sprintf("\n\nTrait %s - %d extreme values flagged (unique_IDs starting with 'P' and 'F1' key
               tr, length(extreme)))

## 3. save BEFORE plot
p_before <- ggplot(dat, aes(x = "", y = .data[[tr]])) +
  geom_boxplot(fill = "#0c4c8a") +
  theme_minimal() +
  ggtitle(glue::glue("{tr}: before"))

## 4. remove those extremes unless ID starts with P or F1
dat2 <- dat %>%
  mutate(
    {{ tr }} := if_else(
      .data[[tr]] %in% extreme & !grepl("^(P|F1)", unique_ID),
      NA_real_,
      .data[[tr]]
    )
  )

## 5. AFTER plot
p_after <- ggplot(dat2, aes(x = "", y = .data[[tr]])) +
  geom_boxplot(fill = "#0c4c8a") +
  theme_minimal() +
  ggtitle(glue::glue("{tr}: after"))

## 6. sanity-check: Rosner should find zero now
if (k > 0) {

  # remove the parent and F1 observations from the data before testing
  vec_keep <- dat2 %>%
  filter(!grepl("^(P|F1)", unique_ID)) %>%
  pull({{ tr }})

  remaining <- rosnerTest(vec_keep,
                           k = length(extreme))$n.outliers
  if (remaining > 0)

```

```

    warning(glue::glue("{tr}: {remaining} outlier(s) still present"))
  }

  ## 7. store updated data and plots
  acc$data <- dat2
  acc$plots[[tr]] <- p_before | p_after
  acc
})

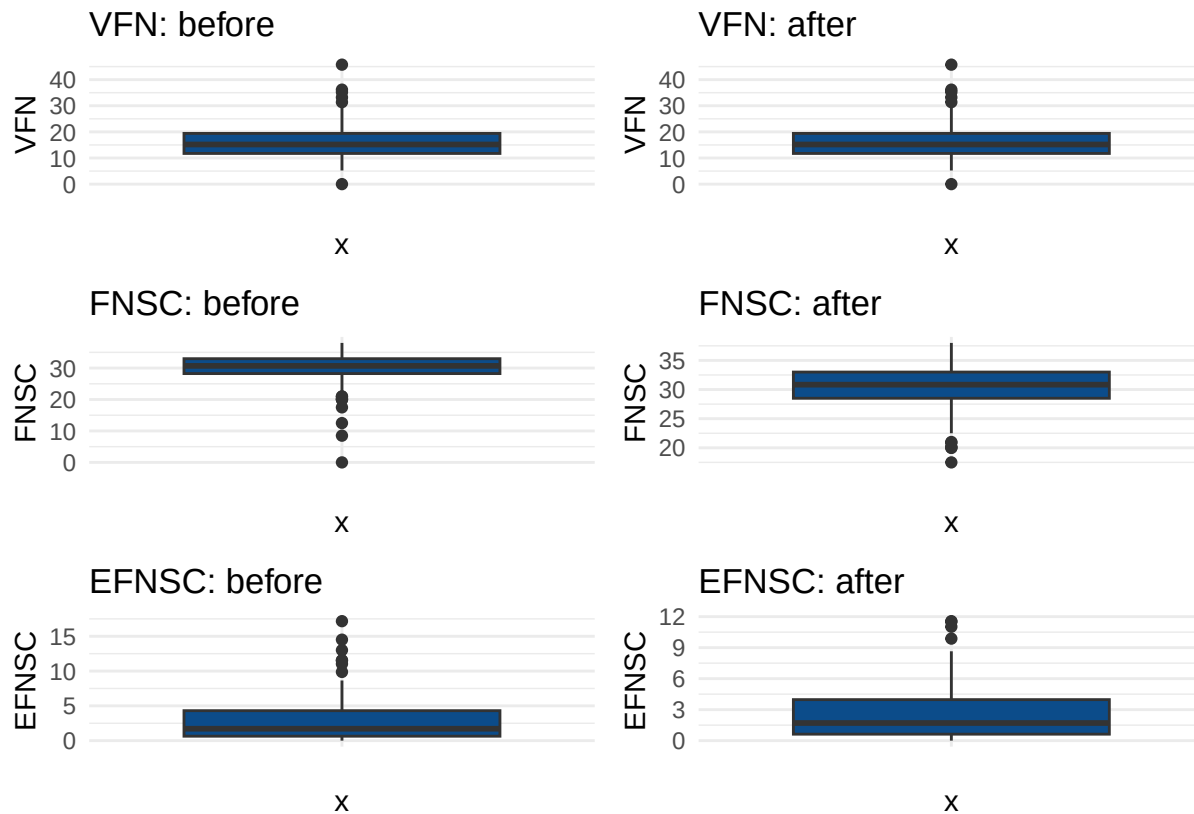
##
##
## Trait VFN - 0 extreme values flagged (unique_IDs starting with 'P' and 'F1' kept)
##
##
## Trait FNSC - 3 extreme values flagged (unique_IDs starting with 'P' and 'F1' kept)
## Warning in rosnerTest(vec_keep, k = length(extreme)): 3 observations with
## NA/NaN/Inf in 'x' removed.
##
##
## Trait EFNSC - 3 extreme values flagged (unique_IDs starting with 'P' and 'F1' kept)
## Warning in rosnerTest(vec_keep, k = length(extreme)): 3 observations with
## NA/NaN/Inf in 'x' removed.
nectar_by_ID <- result$data
plots <- result$plots      # list of before/after panels

# combine all plots into grid
wrap_plots(plots, nrow = length(traits)) # grid of all traits

## Warning: Removed 3 rows containing non-finite outside the scale range
## (`stat_boxplot()`).

## Warning: Removed 3 rows containing non-finite outside the scale range
## (`stat_boxplot()`).

```



Combine and write data

```
data <- inflor_by_ID %>%
  full_join(redarea_by_ID, by="unique_ID") %>%
  full_join(flo_morph_by_ID, by="unique_ID") %>%
  full_join(nectar_by_ID, by="unique_ID")

write.csv(data, "~/Dropbox/Costus/costus-genetic-mapping/phenotype/results/processed_data/phenotypic_data.csv")
```

Exclude parents and F1s

```
#exclude parents and F1s
inflor_by_ID_filtered <- inflor_by_ID %>%
  filter(!grepl("^F1|P", unique_ID))

redarea_by_ID_filtered <- redarea_by_ID %>%
  filter(!grepl("^F1|P", unique_ID))

flo_morph_by_ID_filtered <- flo_morph_by_ID %>%
  filter(!grepl("^F1|P", unique_ID))

nectar_by_ID_filtered <- nectar_by_ID %>%
  filter(!grepl("^F1|P", unique_ID))

data <- inflor_by_ID_filtered %>%
```

```
full_join(redarea_by_ID_filtered, by="unique_ID") %>%  
full_join(flo_morph_by_ID_filtered, by="unique_ID") %>%  
full_join(nectar_by_ID_filtered, by="unique_ID")  
  
write.csv(data, "~/Dropbox/Costus/costus-genetic-mapping/phenotype/results/processed_data/phenotypic_da
```